



CENTRO UNIVERSITÁRIO EUROAMERICANO

ANA LUIZA FERNANDES NOVAES

EMERSON JÚNIOR DE SANTANA OLIVEIRA LEITE

MARIA LUIZA VICENTE DA SILVA LACERDA

MARCELO LEITÃO E SILVA FILHO

STHEFANNY RAMOS PARRINI

YASMIN GOMES DE MATOS

**PROJETO INTEGRADOR: ARQUITETURA E MODELAGEM DO
SISTEMA GLOWUP**

BRASÍLIA

2026

ANA LUIZA FERNANDES NOVAES

EMERSON JÚNIOR DE SANTANA OLIVEIRA LEITE

MARIA LUIZA VICENTE DA SILVA LACERDA

MARCELO LEITÃO E SILVA FILHO

STHEFANNY RAMOS PARRINI

YASMIN GOMES DE MATOS

**PROJETO INTEGRADOR: ARQUITETURA E MODELAGEM DO
SISTEMA GLOWUP**

Documento técnico e modelagem do **funcionamento** do sistema de E-commerce, apresentado como requisito parcial para obtenção de nota na disciplina do **Projeto Integrador De Programação Estruturada**, turmas ADS03M1 e SI03M1, dos cursos de Análise e Desenvolvimento de Sistemas e Sistemas de Informação, pelo Centro Universitário Euroamericano – UNIEURO.

Orientador: Prof. Paulo Augusto

BRASÍLIA

2026

SUMÁRIO

1.OBJETIVO GERAL	6
1.1. Atores do sistema	6
2. OBJETIVO ESPECÍFICO.....	6
3. LEVANTAMENTO DE REQUISITOS.....	7
3.1. Requisitos funcionais	7
3.2. Requisitos não funcionais.....	7
4. DIAGRAMA DE CASOS DE USO	8
4.1. Introdução e contextualização.....	8
5. HISTÓRIAS DE USUÁRIO.....	9
5.1. Histórias de gateway de pagamento	13
6. DIAGRAMA DE SEQUÊNCIA.....	14
6.1. Introdução e Contextualização	14
6.2. Documentação do diagrama de sequência – fluxo principal.....	17
6.2.1. Atores envolvidos.....	17
6.2.2. Participantes (lifelines).....	17
6.2.3. Descrição detalhada do fluxo	18
6.2.3.1 Fase de Cadastro.....	18
6.2.3.2. Fase de login.....	19
6.2.3.4. Fase de navegação e adição de itens ao carrinho	20
6.2.3.5. Fase de finalização do pedido	22
6.3. Documentação do diagrama de sequência – fluxos alternativos (exceções).....	23
6.3.1. Processo de cadastro.....	23

6.3.2. Processo de login.....	23
6.3.3. Processo de edição de perfil	24
6.3.4. Processo de busca e catálogo	24
6.3.5. Processo de adição ao carrinho	25
6.3.5.1. Erros de autenticação	25
6.3.5.2. Erros de disponibilidade (estoque).....	25
6.3.6. Processo de finalização de pedido.....	25
6.3.7. Processo de pagamento e confirmação.....	26
7. BACKLOG ESTRUTURADO DO PROJETO	27
7.1. Priorização de Desenvolvimento.....	27
7.2. Justificativa das prioridades	27
7.2.1 Fase P0 - Alicerce Técnico.....	27
7.2.2 Fase P1 - Núcleo de Operação	28
7.2.3 Fase P2 - Refinamento e Robustez.....	28
8. TELAS.....	28
9. CÓDIGO	34
10. MANUAL DO USUÁRIO - SISTEMA GLOWUP (E-COMMERCE)	48
10.1. Visão Geral do Sistema	48
10.1.1. Como Executar (Primeiros Passos).....	48
10.2. Acesso ao Sistema e Segurança.....	48
10.2.1. Criando sua Conta (Opção 1)	48
10.2.2. Fazer Login (Opção 2)	49
10.3. Navegando e Comprando (Painel do Cliente).....	49
10.4.. Checkout (Finalizando o Pedido).....	49

10.5. Pós-Venda e Edição de Perfil	50
10.6. Notas Técnicas e Sistema Anti-Crash	50
PLANNER DOCUMENTAÇÃO	51
PLANNER CODIFICAÇÃO DO SISTEMA	53

1. OBJETIVO GERAL

O sistema GlowUp foi idealizado como uma plataforma de e-commerce sólida e confiável, voltada para a venda de cosméticos. O principal objetivo da solução é proporcionar ao consumidor uma experiência de compra intuitiva, enquanto assegura que todas as regras de negócio, como controle de estoque e validação financeira, sejam estritamente atendidas em segundo plano.

1.1. Atores do sistema

Para que o sistema funcione adequadamente, foram identificados os Atores que se comunicam diretamente ou indiretamente com a plataforma:

- **Novo Cliente:** É o visitante que visita a loja pela primeira vez e precisa passar pelo processo de registro para se tornar um usuário ativo.
- **Cliente:** é o protagonista do sistema. Depois de fazer o login, ele é responsável por navegar pelo catálogo, montar seu carrinho e finalizar os pedidos.
- **Gateway de Pagamento:** Funciona como um participante secundário (sistema externo). Ele não é um indivíduo, mas o serviço encarregado de processar transações financeiras (cartão, Pix, boleto) e retornar à aprovação ou rejeição para nossa loja.

2. OBJETIVO ESPECÍFICO

Para cumprir a proposta do sistema GlowUp, os seguintes objetivos específicos foram estabelecidos:

- **Aplicar os conceitos da disciplina de Programação Estruturada** na construção do núcleo de um sistema de gerenciamento para uma loja de cosméticos, organizando o código por meio de funções e modularização.
- **Implementar o cadastro**, listagem, atualização e remoção de produtos em memória, utilizando estruturas de dados adequadas (como structs e vetores).
- **Garantir o controle rigoroso do fluxo de execução do sistema e a validação dos dados** inseridos pelo usuário utilizando estruturas condicionais e de repetição.
- **Desenvolver a lógica do carrinho e do registro de vendas**, com foco no cálculo automático do valor total das compras com base nos itens selecionados.
- **Estruturar o controle inicial de estoque** por meio da manipulação de dados em memória, garantindo a baixa automática dos produtos após cada venda.

- **Desenvolver uma interface de navegação fluida** que permita ao cliente localizar os produtos por nome ou categoria de forma simplificada e intuitiva.
- **Implementar um sistema de autenticação e segurança de acesso** para proteger os dados cadastrais e o histórico de compras dos utilizadores.
- **Assegurar a integridade avançada do inventário**, estabelecendo verificações de disponibilidade de estoque em tempo real antes da aprovação de qualquer pagamento.
- **Automatizar o ciclo financeiro do sistema**, integrando-o a gateways de pagamento externos para o processamento de transações via PIX, boleto e cartões.
- **Otimizar a logística e o pós-venda por meio da integração de APIs** para o cálculo de frete, acompanhamento do status da entrega e avaliação dos produtos comprados.

3. LEVANTAMENTO DE REQUISITOS

3.1. Requisitos funcionais

Em relação às Funcionalidades Principais (Requisitos Funcionais), o sistema foi projetado para abranger todo o processo de vendas:

- **Gestão de Conta:** o sistema possibilita ao usuário criar uma conta, realizar login de maneira segura e administrar suas informações pessoais e endereços de entrega.
- **Experiência de Compra:** O cliente tem a opção de procurar produtos pelo nome ou categoria, visualizar especificações técnicas e adicionar itens ao carrinho de compras, no qual o sistema calcula automaticamente os subtotais.
- **Checkout e Pagamento:** Ao finalizar o pedido, o sistema consolida a compra, permite a aplicação de cupons de desconto, calcula o valor do frete e se conecta ao Gateway para realizar o processamento do pagamento.
- **Pós-Venda:** Depois de efetuar a compra, o cliente tem a possibilidade de acompanhar o status do pedido (em processamento, enviado, entregue) e, por fim, deixar uma avaliação sobre os produtos comprados.

3.2. Requisitos não funcionais

Para assegurar a qualidade e a viabilidade do negócio, o sistema também cumpre regras rigorosas e Requisitos Não Funcionais:

- **Restrições de Negócio:** Para efetuar compras, o usuário precisa ter pelo menos 16 anos. Ademais, por motivos logísticos, o sistema impede o uso de endereços de entrega fora do Brasil.
- **Integridade e Estoque:** O sistema verifica a disponibilidade dos produtos em tempo real antes de aprovar qualquer pagamento, prevenindo a venda de itens esgotados.
- **Integrações:** a estrutura do sistema depende da interação com APIs externas para calcular o frete (Correios/Transportadoras) e processar pagamentos, o que requer que essas conexões estejam sempre disponíveis.

4. DIAGRAMA DE CASOS DE USO

4.1. Introdução e contextualização

O diagrama abaixo apresenta os **Casos de Uso do sistema de E-commerce**,

demonstrando as principais funcionalidades que o sistema oferece e a forma como os usuários interagem com ele. A finalidade deste modelo é estabelecer o alcance do sistema sob a perspectiva do usuário.

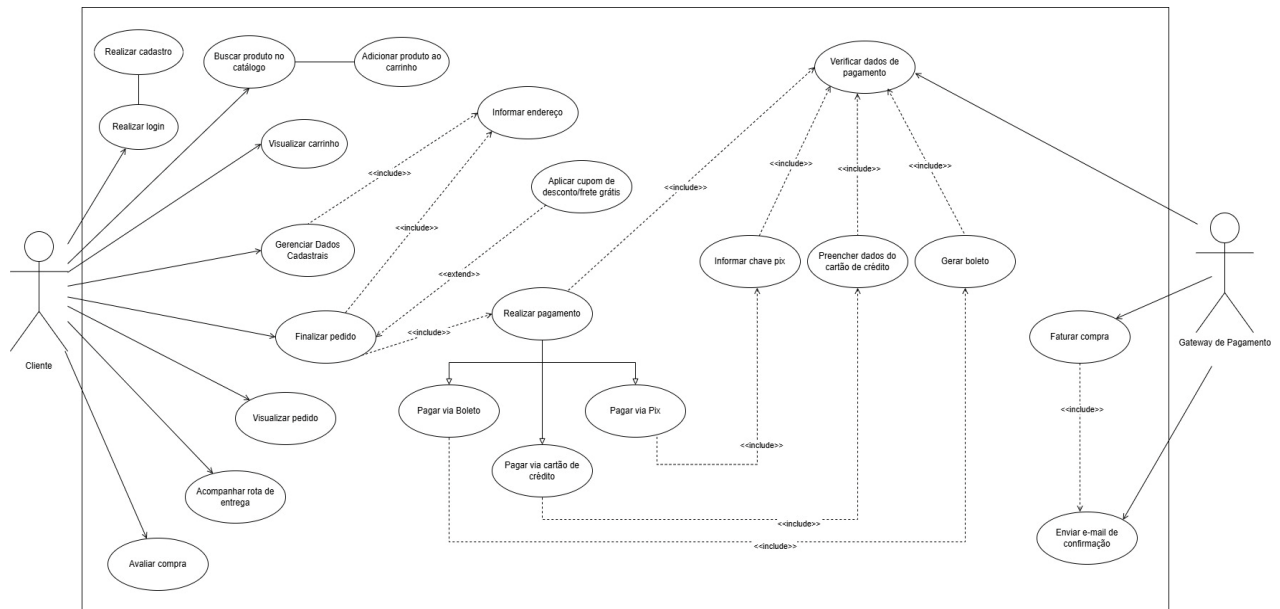
Com base no documento, segue uma introdução e contextualização específicas para o Diagrama de Casos de Uso:

Os **atores** envolvidos neste diagrama são:

- **Cliente:** O ator principal que usa o sistema para executar tarefas como fazer login, procurar produtos, adicionar itens ao carrinho, finalizar um pedido, visualizar compras anteriores e avaliar suas aquisições.
- **Gateway de Pagamento:** um ator secundário que simboliza o sistema externo encarregado de processar a transação financeira, emitir a fatura da compra e enviar o e-mail de confirmação.

Este diagrama descreve a trajetória do cliente, desde a autenticação até a conclusão da compra, representando as etapas fundamentais, como a finalização do pedido, a execução do pagamento e as validações de dados requeridas.

Figura 1 – Diagrama de Casos de Uso



Fonte: Elaborado pelos autores (2026). Disponível em: <https://drive.google.com/file/d/1KQM96jckGs3AxmttNA14MZpl62XpYlAZ/view?usp=sharing>

5. HISTÓRIAS DE USUÁRIO

HU01 – Realizar Cadastro	
Campo	Descrição
Descrição	Como novo cliente, quero criar uma conta no sistema para poder realizar compras e ter meus dados salvos para o futuro.
Pré-condição	Usuário não logado no sistema.
Fluxo Principal	1. Clicar na opção "Cadastrar" ou "Criar Conta" 2. Preencher formulário com dados pessoais (Nome, CPF, E-mail, Senha) 3. Confirmar o cadastro. 4. Sistema confirma a criação da conta.
Fluxo Alternativo	E-mail ou CPF já cadastrados → Sistema exibe mensagem de erro.
Critérios de aceitação	Cliente cadastrado e apto a fazer login.

Fonte: Elaborado pelos autores (2026).

HU02 – Realizar Login	
Campo	Descrição
Descrição	Como cliente, quero realizar login no sistema para acessar minha conta e realizar compras
Pré-condição	Cliente cadastrado
Fluxo Principal	1. Informar e-mail e senha. 2. Sistema valida credenciais. 3. Sistema concede acesso.
Fluxo Alternativo	Dados inválidos → Sistema exibe mensagem de erro.
Critérios de aceitação	Cliente autenticado no sistema.

Fonte: Elaborado pelos autores (2026).

HU03 – Gerenciar Dados Cadastrais	
Campo	Descrição
Descrição	Como cliente logado , quero visualizar e editar minhas informações pessoais e de endereço para manter meu cadastro atualizado.
Pré-condição	Cliente autenticado (logado) no sistema.
Fluxo Principal	1. Acessar a área "Minha Conta" ou "Perfil". 2. Selecionar a opção de editar dados. 3. Alterar as informações desejadas (ex: endereço, telefone, senha, e-mail). 4. Salvar alterações.
Fluxo Alternativo	Dados inválidos → Sistema impede a alteração e alerta o usuário.
Critérios de aceitação	Dados do cliente atualizados no banco de dados.

Fonte: Elaborado pelos autores (2026).

HU04 – Buscar Produto no Catálogo	
Campo	Descrição
Descrição	Como cliente, quero buscar produtos no catálogo para encontrar itens de interesse.
Pré-condição	Sistema disponível
Fluxo Principal	1. Cliente digita termo de busca. 2. Sistema exibe resultados.

Pós-condição	Lista de produtos exibida.
---------------------	----------------------------

Fonte: Elaborado pelos autores (2026).

HU05 – Adicionar Produto ao Carrinho	
Campo	Descrição
Descrição	Como cliente, quero adicionar produtos ao carrinho para comprá-los posteriormente.
Pré-condição	Produto disponível.
Fluxo Principal	1. Selecionar produto. 2. Clicar em “<i>Adicionar ao carrinho</i>”. 3. Sistema atualiza carrinho.
Fluxo Alternativo	A1: Usuário não autenticado ou não cadastrado → Sistema interrompe a ação e redireciona o usuário para a tela de cadastro/login. A2: Produto indisponível → Sistema informa erro.
Pós-condição	Produto incluído no carrinho.

Fonte: Elaborado pelos autores (2026).

HU06 – Finalizar Pedido	
Campo	Descrição
Descrição	Como cliente, quero finalizar meu pedido para concluir minha compra.
Pré-condição	Carrinho com itens
Fluxo Principal	1. Acessar carrinho. 2. Confirmar itens. 3. Informar endereço. 4. Aplicar cupom(opcional). 5. Selecionar forma de pagamento.
Fluxo Alternativo	A1: Carrinho vazio → Cliente tenta concluir a compra sem itens; a interface bloqueia o avanço para a tela de pagamento. A2: Estoque insuficiente → Sistema valida que não há estoque para as quantidades escolhidas e o pedido é interrompido. A3: Produto indisponível → Sistema informa erro.

Pós-condição	Pedido registrado com sucesso.
---------------------	--------------------------------

Fonte: Elaborado pelos autores (2026).

HU07 – Realizar Pagamento	
Campo	Descrição
Descrição	Como cliente, quero realizar o pagamento para confirmar minha compra.
Pré-condição	Pedido criado
Fluxo Principal	- Escolher método de pagamento. - PIX - Boleto - Cartão de crédito ou débito - Informar dados necessários. - Sistema envia dados ao gateway.
Pós-condição	Pagamento aprovado ou recusado.

Fonte: Elaborado pelos autores (2026).

HU08– Acompanhar Entrega	
Campo	Descrição
Descrição	Como cliente, quero acompanhar a rota de entrega para saber quando meu pedido chegará.
Pré-condição	Pedido enviado
Fluxo Principal	- Acessar área de pedidos. - Visualizar status de entrega.
Pós-condição	Status exibido ao cliente.

Fonte: Elaborado pelos autores (2026).

HU09 – Avaliar Compra	
Campo	Descrição
Descrição	Como cliente, quero avaliar minha compra para registrar minha experiência.
Pré-condição	Pedido entregue
Fluxo Principal	- Selecionar pedido. - Atribuir nota e comentário. - Confirmar avaliação.
Pós-condição	Avaliação registrada no sistema.

Fonte: Elaborado pelos autores (2026).

5.1. Histórias de gateway de pagamento

HG01 – Verificar Dados de Pagamento	
Campo	Descrição
Descrição	Como gateway de pagamento, quero validar os dados financeiros recebidos para autorizar ou negar a transação.
Pré-condição	Dados recebidos do sistema
Fluxo Principal	1. Validar dados. 2. Autorizar ou recusar. 3. Retornar status ao sistema.
Pós-condição	Transação com status definido.

Fonte: Elaborado pelos autores (2026).

HG02 – Faturar compra	
Campo	Descrição
Descrição	Como sistema, quero faturar a compra para registrar oficialmente a venda.
Pré-condição	Pagamento aprovado
Fluxo Principal	1. Registrar venda. 2. Atualizar status do pedido. 3. Enviar e-mail de confirmação.
Pós-condição	Pedido faturado e confirmado ao cliente.

Fonte: Elaborado pelos autores (2026).

6. DIAGRAMA DE SEQUÊNCIA

6.1. Introdução e Contextualização

O **Diagrama de Sequência** a seguir descreve a sequência temporal das interações necessárias para o funcionamento do sistema de comércio eletrônico. Este diagrama visa complementar os modelos anteriores, detalhando a jornada completa do usuário e esclarecendo a sequência de mensagens trocadas entre os atores e os componentes do sistema.

Os atores envolvidos neste fluxo são:

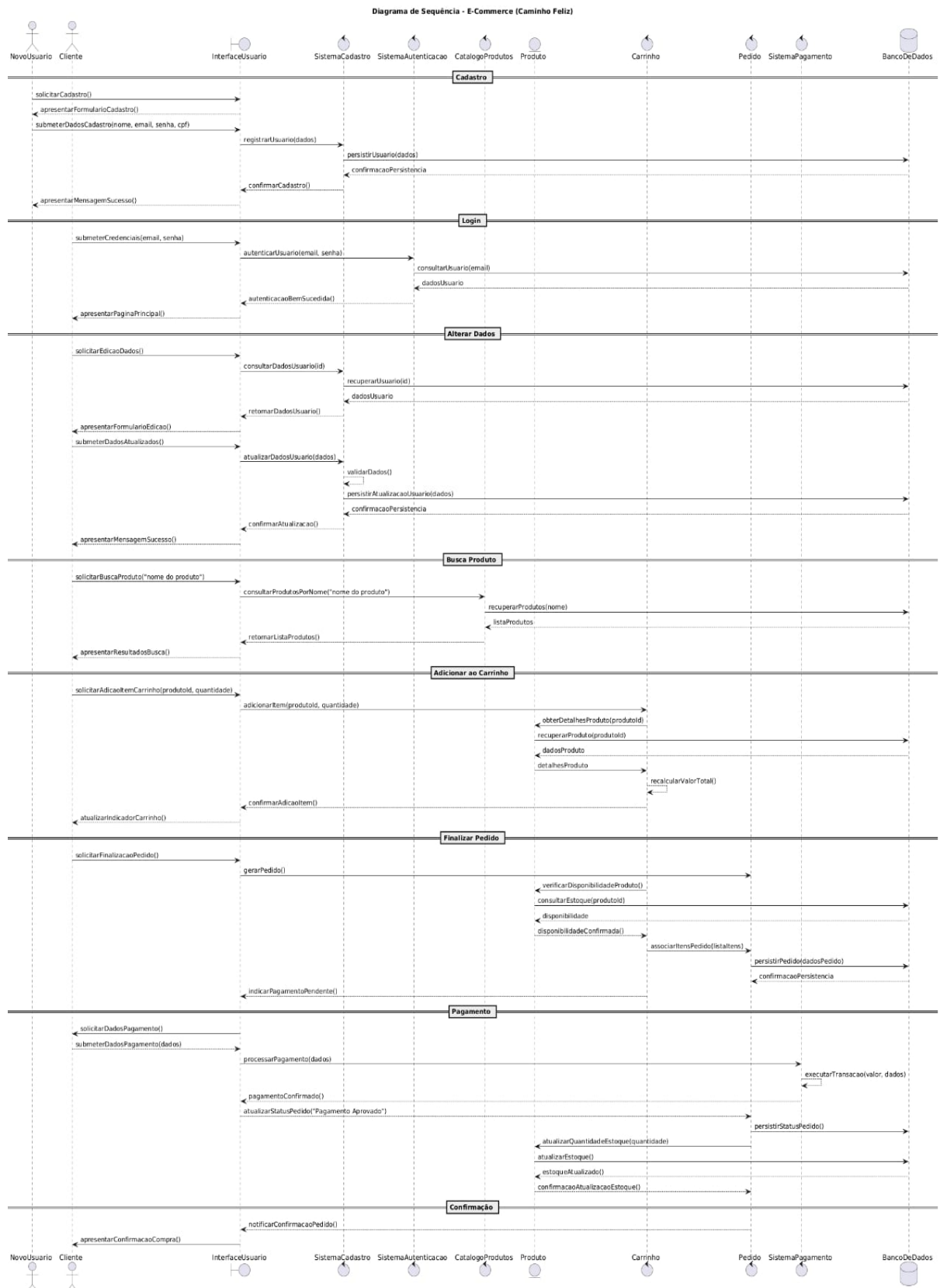
- **NovoCliente:** Refere-se a um usuário que ainda não tem cadastro e começa o procedimento de criação de conta.
- **Cliente:** Refere-se a um usuário previamente cadastrado que se autentica para efetuar compras.

Este diagrama representa os fluxos completos de:

1. **Cadastro** de um novo usuário.
2. **Login** de um cliente já cadastrado.
3. **Navegação** e inclusão de itens no carrinho.
4. **Finalização do pedido**, que resulta no processamento do pagamento.

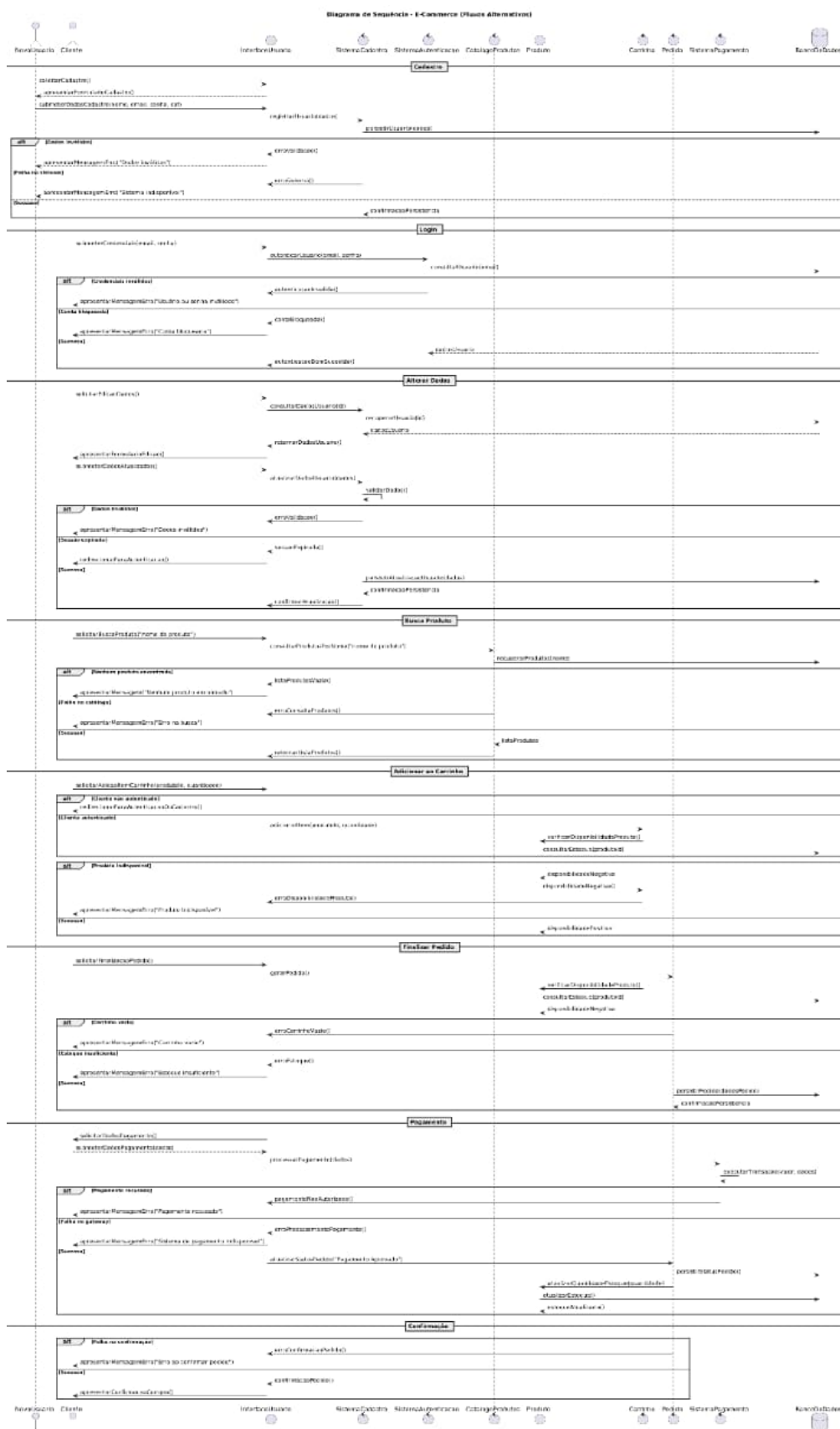
Ele ilustra como os atores interagem com as diferentes *lifelines* (componentes do sistema), como a **:InterfaceUsuario**, o **:SistemaCadastro**, o **:SistemaAutenticacao**, o **:CatalogoProdutos**, o **:Carrinho**, o **:Pedido** e o **:SistemaPagamento**

Figura 2 – Diagrama de Sequência – Fluxo Principal



Fonte: Elaborado pelos autores (2026). Disponível em: <https://drive.google.com>

Figura 3 – Diagrama de Sequência – Fluxos Alternativos



Fonte: Elaborado pelos autores (2026). Disponível em: <https://drive.google.com>

6.2. Documentação do diagrama de sequência – fluxo principal

6.2.1. Atores envolvidos

Quadro 01 – Atores Envolvidos no Processo

Ator	Descrição
NovoCliente	Representa um usuário que ainda não possui cadastro e deseja criar uma conta no sistema.
Cliente	Representa um usuário já registrado e autenticado, que interage com as funcionalidades de compra e perfil.

Fonte: Elaborado pelos autores (2026).

6.2.2. Participantes (lifelines)

Quadro 02 – Participantes (Lifelines) e Estereótipos

Participante	Estereótipo	Função Principal	Exemplos de Mensagens (Entrada/Saída)
:InterfaceUsuario	Boundary	Camada de apresentação e interação direta com o ator (UI/UX e Front-end).	solicitarCadastro(), submeterCredenciais(), apresentarPaginaPrincipal()
:SistemaCadastro	Control	Orquestra a lógica de criação e atualização de contas.	registrarUsuario(), atualizarDadosUsuario()
:SistemaAutenticacao	Control	Gerencia a lógica de autenticação de usuários e validação de credenciais.	autenticarUsuario(), consultarUsuario()
:CatalogoProdutos	Control	Controla a lógica de busca, filtragem e recuperação de informações de produtos.	consultarProdutosPorNome()

:Produto	Entity	Entidade de dados persistente que armazena informações de um item comercializável.	obterDetalhesProduto(), atualizarQuantidadeEstoque()
:Carrinho	Control	Gerencia os itens selecionados para compra; orquestrador temporário.	adicionarItem(), recalcularTotal()
:Pedido	Entity	Entidade de dados persistente que armazena todas as informações de uma transação de compra.	gerarPedido(), associarItensPedido()
:SistemaPagamento	Control	Serviço externo (gateway) para processamento e validação de transações financeiras.	processarPagamento(), executarTransacao()
:BancoDeDados	Entity	Repositório persistente para armazenamento de todos os dados do sistema.	persistirUsuario(), recuperarProdutos(), consultarEstoque()

Fonte: Elaborado pelos autores (2026).

6.2.3. Descrição detalhada do fluxo

6.2.3.1 Fase de Cadastro

Quadro 03 – Fluxo de Mensagens: Cadastro

Seq.	De	Para	Mensagem	Descrição
1	NovoCliente	:InterfaceUsuario	solicitarCadastro()	O NovoCliente solicita abertura de formulário.
1.1	:InterfaceUsuario	NovoCliente	apresentarFormularioCadastro()	A interface exibe o formulário para o usuário preencher.
2	NovoCliente	:InterfaceUsuario	submeterDadosCadastro (nome, email, senha, cpf)	O NovoCliente insere suas informações e submete.

2.1	:InterfaceUsuario	:SistemaCadastro	registrarUsuario(dados)	A interface envia os dados para o sistema de cadastro.
2.2	:SistemaCadastro	:BancoDeDados	persistirUsuario(dados)	O sistema salva os dados permanentemente.
2.3	:BancoDeDados	:SistemaCadastro	confirmacaoPersistencia()	O banco confirma o salvamento com sucesso.
2.4	:SistemaCadastro	:InterfaceUsuario	confirmarCadastro()	O sistema avisa à interface que o cadastro foi concluído.
2.5	:InterfaceUsuario	NovoUsuario	apresentarMensagemSucesso()	Feedback visual de sucesso para o usuário.

Fonte: Elaborado pelos autores (2026).

6.2.3.2. Fase de login

Quadro 04 – Fluxo de Mensagens: Autenticação

Seq.	De	Para	Mensagem	Descrição
3	Cliente	:InterfaceUsuario	submeterCredenciais(email, senha)	O Cliente insere suas credenciais de acesso.
3.1	:InterfaceUsuario	:SistemaAutenticacao	autenticarUsuario(email, senha)	A interface envia as credenciais para o sistema de autenticação.
3.2	:SistemaAutenticacao	:BancoDeDados	consultarUsuario(email)	Busca as credenciais no banco de dados.
3.3	:BancoDeDados	:SistemaAutenticacao	dadosUsuario	Retorna as informações para comparação.
3.4	:SistemaAutenticacao	:InterfaceUsuario	autenticacaoBemSucedida()	O sistema confirma que o login é válido. e autoriza a entrada.
3.5	:InterfaceUsuario	Cliente	apresentarPaginaPrincipal()	A interface exibe a página principal do e-commerce para o Cliente logado.

6.2.3.3. Fase de alteração de dados cadastrais

Quadro 05 – Fluxo de Mensagens: Alterar Dados

Seq.	De	Para	Mensagem	Descrição
4	Cliente	:InterfaceUsuario	clicarEmMeusDados()	O cliente logado solicita o acesso à edição de seu perfil.
4.1	:InterfaceUsuario	:SistemaCadastro	solicitarDadosCliente(id)	A interface requisita os dados atuais do cliente ao sistema.
4.2	:SistemaCadastro	:BancoDeDados	recuperarUsuario(id)	Busca as informações no banco.
4.3	:SistemaCadastro	:InterfaceUsuario	:retornarDadosUsuario()	Devolve os dados para preencher a tela.
5	Cliente	:InterfaceUsuario	submeterDadosAtualizados()	O usuário envia as novas informações.
5.1	:InterfaceUsuario	:SistemaCadastro	atualizarDadosUuario(dados)	Processa a atualização no controle.
5.2	:SistemaCadastro	:SistemaCadastro	validarDados()	O sistema verifica internamente a validade das novas informações.
5.3	:SistemaCadastro	:BancoDeDdados	persistirAtualizacaoUsuario()	Sobrescreve os dados antigos no banco.

6.2.3.4. Fase de navegação e adição de itens ao carrinho

Quadro 06 – Fluxo de Mensagens: Fluxo de Compra

Seq.	De	Para	Mensagem	Descrição
------	----	------	----------	-----------

6	Cliente	:InterfaceUsuario	solicitarBuscaProduto()	O Cliente inicia uma busca por um produto.
6.1	:InterfaceUsuario	:CatalogoProdutos	consultarProdutosPorNome()	A interface solicita ao catálogo a lista de produtos com a palavra-chave.
6.2	:CatalogoProdutos	:BancoDeDados	recuperarProdutos(nome)	Busca registros correspondentes no BD.
7	Cliente	:InterfaceUsuario	solicitarAdicaoItemCarrinho()	O Cliente seleciona um produto e adiciona ao carrinho.
7.1	:InterfaceUsuario	:Carrinho	adicionarItem(id, qtd)	Carrinho inicia a lógica de inclusão.
7.2	:Carrinho	:Produto	obterDetalhesProduto(id)	Busca preço e nome do produto.
7.3	:Carrinho	:Carrinho	recalcularTotal()	O Carrinho atualiza seu valor total.
8	Cliente	:InterfaceUsuario	solicitarFinalizacaoPedido()	Inicia o checkout.
8.1	:InterfaceUsuario	:Pedido	gerarPedido()	Cria a instância do pedido.
8.2	:Pedido	:Produto	verificarDisponibilidadeProduto()	Checa se há estoque antes de pagar.
8.3	:Pedido	:BancoDeDados	verificarDisponibilidadeProduto()	Salva o pedido como pendente no banco.

Fonte: Elaborado pelos autores (2026).

6.2.3.5. Fase de finalização do pedido

Quadro 07 – Fluxo de Mensagens: Finalização e Pagamento

Seq.	De	Para	Mensagem	Descrição
9	Cliente	:InterfaceUsuario	submeterDadosPagamento()	Envia dados do cartão ou PIX.
9.1	:InterfaceUsuario	:SistemaPagamento	processarPagamento(dados)	Envia os dados para o gateway.
9.2	:SistemaPagamento	:SistemaPagamento	executarTransacao()	Comunicação interna com operadoras financeiras.
9.3	:SistemaPagamento	:Pedido	atualizarStatusPedido("Aprovado")	Muda o status do pedido após confirmação.
9.4	:Pedido	:Produto	atualizarQuantidadeEstoque(qtd)	Reduz a quantidade de estoque do produto.
10	:Pedido	:InterfaceUsuario	notificarConfirmacaoPedido()	Envia o sinal de finalização para a tela.
10.1	:InterfaceUsuario	Cliente	apresentarConfirmacaoCompra()	Exibe a mensagem final de agradecimento e sucesso.

Fonte: Elaborado pelos autores (2026).

6.3. Documentação do diagrama de sequência – fluxos alternativos (exceções)

Para garantir a robustez do sistema, foi modelado também o diagrama de fluxos alternativos, demonstrando como o sistema reage a falhas de validação, problemas de conexão ou regras de negócio não atendidas (tratamento de erros).

6.3.1. Processo de cadastro

Quadro 08 – Fluxo de Mensagens: Alternativas de Cadastro

Seq.	De	Para	Mensagem	Descrição
2.4a	:SistemaCadastro	:InterfaceUsuario	erroValidacao()	O sistema detecta que os dados enviados são inválidos ou já existem no banco.
2.5a	:InterfaceUsuario	NovoCliente	apresentarMsgErro(“Dados inválidos”)	A interface apresenta um alerta de erro de validação ao usuário.
2.4b	:SistemaCadastro	:InterfaceUsuario	erroSistema()	Falha técnica interna ou perda de conexão com o banco de dados.
2.5b	:InterfaceUsuario	NovoCliente	apresentarMsgErro(“Sistema indisponível”)	O usuário é informado que o serviço está momentaneamente fora do ar.

Fonte: Elaborado pelos autores (2026)

6.3.2. Processo de login

Quadro 09 – Fluxo de Mensagens: Alternativas de Login

Seq.	De	Para	Mensagem	Descrição
3.4	:SistemaAutenticacao	:InterfaceUsuario	autenticacaoInvalida()	As credenciais fornecidas não correspondem a nenhum registro ativo.
3.5a	:InterfaceUsuario	Cliente	apresentarMsgErro(“Usuário ou senha inválidos”)	A interface solicita que o cliente verifique os dados de acesso.

3.4b	:SistemaAutenticacao	:InterfaceUsuario	contaBloqueada()	Tentativa de acesso em uma conta que possui restrições de segurança.
3.5b	:InterfaceUsuario	Cliente	apresentarErro("Conta bloqueada")	Informação de bloqueio de conta é apresentada ao cliente.

Fonte: Elaborado pelos autores (2026).

6.3.3. Processo de edição de perfil

Quadro 10 – Fluxo de Mensagens: Alternativas de Alterar Dados

Seq.	De	Para	Mensagem	Descrição
5.4a	:SistemaCadastro	:InterfaceUsuario	erroValidacao()	Erro detectado após a submissão das novas informações cadastradas.
5.5a	:InterfaceUsuario	Cliente	apresentarMsgErro("Dados inválidos")	Notificação de que as alterações não puderam ser salvas.
5.4b	:SistemaCadastro	:InterfaceUsuario	sessaoExpirada()	O token de segurança expirou durante o tempo de edição do formulário.
5.5b	:InterfaceUsuario	Cliente	redirecionarLogin()	O sistema encerra a sessão atual e solicita novo login por segurança.

Fonte: Elaborado pelos autores (2026)

6.3.4. Processo de busca e catálogo

Quadro 11 – Fluxo de Mensagens: Alternativas de Busca de Produto

Seq.	De	Para	Mensagem	Descrição
6.3a	:CatalogoProdutos	:InterfaceUsuario	listaProdutosVazia()	O termo pesquisado não possui produtos correspondentes no estoque.

6.4a	:InterfaceUsuario	Cliente	apresentarMsg(“Nenhum produto encontrado”)	Mensagem informativa exibida na área de resultados da busca.
6.3b	:CatalogoProdutos	:InterfaceUsuario	erroConsultaProduto()	Falha de comunicação técnica com o servidor de banco de produtos.
6.4b	:InterfaceUsuario	Cliente	apresentarMsgErro(“Erro na busca”)	Alerta de erro genérico no sistema de busca de produtos.

Fonte: Elaborado pelos autores (2026).

6.3.5. Processo de adição ao carrinho

Esta seção detalha as exceções que impedem a inclusão de itens no carrinho, divididas entre validação de acesso e regras de inventário.

6.3.5.1. Erros de autenticação

Quadro 12 – Fluxo de Mensagens: Exceções de Identificação

Seq.	De	Para	Mensagem	Descrição
7.1a	:InterfaceUsuario	Cliente	redirecionarParaIdentificacaoOuCadastro()	Impede a adição ao carrinho se o usuário for um visitante não logado.

Fonte: Elaborado pelos autores (2026).

6.3.5.2. Erros de disponibilidade (estoque)

Quadro 13 – Fluxo de Mensagens: Exceções de Estoque

Seq.	De	Para	Mensagem	Descrição
7.3a	:Carrinho	:InterfaceUsuario	erroEstoque()	O produto ficou indisponível no exato momento da adição ao carrinho.
7.4a	:InterfaceUsuario	Cliente	apresentarMsg(“Produto indisponível”)	Alerta crítico sobre a falta de estoque do item selecionado.

Fonte: Elaborado pelos autores (2026).

6.3.6. Processo de finalização de pedido

Quadro 14 – Fluxo de Mensagens: Alternativas de Finalizar Pedido

Seq.	De	Para	Mensagem	Descrição
8.1a	:Carrinho	:InterfaceUsuario	erroCarrinhoVazio()	O cliente tenta concluir a compra

				sem ter itens selecionados.
8.2a	:InterfaceUsuario	Cliente	apresentarMsg(“Carrinho vazio”)	A interface bloqueia o avanço para a tela de pagamento.
8.2b	:Pedido	:InterfaceUsuario	erroEstoque()	No momento do fechamento, um ou mais itens ficaram sem estoque.
8.3b	:InterfaceUsuario	Cliente	exibirMsgErro(“Estoque insuficiente”)	O pedido é interrompido por inconsistência de inventário.

Fonte: Elaborado pelos autores (2026).

6.3.7. Processo de pagamento e confirmação

Quadro 15– Fluxo de Mensagens: Alternativas de Pagamento e Confirmação

Seq.	De	Para	Mensagem	Descrição
9.3a	:Sistema de Pagamento	:Pedido	pagamentoNaoAutorizado()	O gateway externo negou a transação financeira enviada.
9.4a	:InterfaceUsuario	Cliente	apresentarMsgErro(“Pagamento recusado”)	Sugestão ao cliente para utilizar um novo método de pagamento.
9.3b	:Sistema de Pagamento	:Pedido	erroProcessoPagamento()	Instabilidade na conexão com o sistema de pagamentos externo.
9.4b	:InterfaceUsuario	Cliente	apresentarMsgErro(“Sistema de pagamento indisponível”)	Alerta de indisponibilidade momentânea do serviço financeiro.
10.1a	:Pedido	:InterfaceUsuario	erroConfirmacao()	Erro sistêmico crítico ao gerar o comprovante final da venda.
10.2a	:InterfaceUsuario	Cliente	apresentarMsgErro(“Erro ao confirmar pedido”)	Feedback final de falha no fechamento do pedido no banco de dados.

Fonte: Elaborado pelos autores (2026)

7. BACKLOG ESTRUTURADO DO PROJETO

Este backlog apresenta a organização sistemática das tarefas de desenvolvimento do sistema de E-commerce GlowUp. A estruturação segue uma lógica de dependência técnica em linguagem C, priorizando a base de dados em memória antes das funcionalidades de interface e regras de negócio.

7.1. Priorização de Desenvolvimento

Quadro 16 – Classificação de Tarefas por Nível de Prioridade

Prioridade	Categoria	Tarefa Principal	ID
P0 (Crítica)	Structs e Dados (memória)	Definição de <i>estruturas.h</i> e Vetores Globais	-
P0 (Crítica)	Interfaces de Menus Principais	Estrutura de Menus (Usuário e Cliente)	-
P0 (Crítica)	Autentificação e Perfil	Módulo de Cadastro de Usuário	HU01
P1 (Essencial)	Autentificação e Perfil	Sistema de Login e Autenticação	HU02
P1 (Essencial)	Catálogo e Produtos	Exibição de Catálogo e Mecanismo de Busca	HU04
P1 (Essencial)	Carrinho e Compras	Gerenciamento de Carrinho e Checkout	HU05 e HU06
P1 (Essencial)	Carrinho e Compras	Processamento de Pagamento e Baixa de Estoque	HU07 e HU10
P2 (Desejável)	Autentificação e Perfil	Gerenciar Dados Cadastrais	HU03
P2 (Desejável)	Pós-Venda	Acompanhar Entrega e Avaliar Compra	HU08 e HU09
P2 (Desejável)	Testes	Testes de Limite, Validação e Integração	-

Fonte: Elaborado pelos autores (2026)

7.2. Justificativa das prioridades

7.2.1 Fase P0 - Alicerce Técnico

O desenvolvimento da categoria **Structs e Dados (Memória)** é o pré-requisito para qualquer outra funcionalidade, dado que o sistema opera com armazenamento volátil. O

cadastro inicial é incluído aqui para permitir a existência de dados para teste dos módulos subsequentes, integrando-se rapidamente às **Interfaces e Menus Principais**.

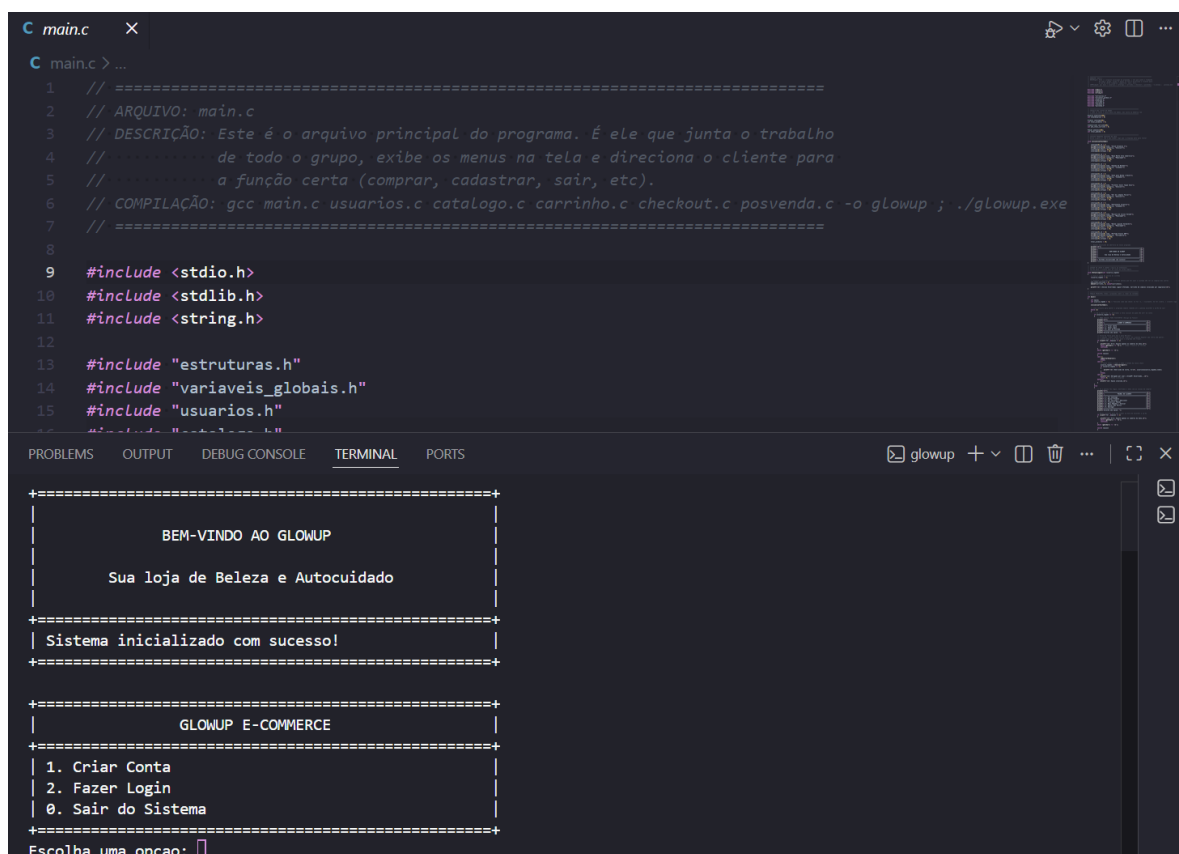
7.2.2 Fase P1 - Núcleo de Operação

Nesta fase, implementa-se o "Caminho Feliz" da aplicação. O foco é garantir que os módulos de **Autenticação e Perfil**, **Catálogo e Produtos**, e **Carrinho e Compras** funcionem em conjunto para que a transação financeira simulada ocorra com a respectiva atualização do inventário.

7.2.3 Fase P2 - Refinamento e Robustez

As tarefas de P2 englobam funcionalidades que agregam valor à experiência, como a edição cadastral e as etapas de **Pós-Venda (HU08 e HU09)**, mas que não impedem a operação básica do e-commerce. Inclui-se aqui a execução da categoria de **Testes** finais para garantir a estabilidade do software contra entradas de dados inválidas.

8. TELAS



```

C main.c x
C main.c > ...
1 // =====
2 // ARQUIVO: main.c
3 // DESCRIÇÃO: Este é o arquivo principal do programa. É ele que junta o trabalho
4 // de todo o grupo, exibe os menus na tela e direciona o cliente para
5 // a função certa (comprar, cadastrar, sair, etc).
6 // COMPILAÇÃO: gcc main.c usuarios.c catalogo.c carrinho.c checkout.c posvenda.c -o glowup ; ./glowup.exe
7 // =====
8
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12
13 #include "estruturas.h"
14 #include "variaveis_globais.h"
15 #include "usuarios.h"

```

```

=====
|                                     |
|          BEM-VINDO AO GLOWUP        |
|                                     |
|          Sua loja de Beleza e Autocuidado          |
|                                     |
|-----|
| Sistema inicializado com sucesso! |
|-----|
|                                     |
|-----|
|          GLOWUP E-COMMERCE          |
|-----|
| 1. Criar Conta                      |
| 2. Fazer Login                      |
| 0. Sair do Sistema                  |
|-----|
Escolha uma opcao: 

```

Link do repositório: <https://github.com/analuizanovaes/GlowUp-Ecommerce>

```

C main.c
C main.c > ...
1 // =====
2 // ARQUIVO: main.c
3 // DESCRIÇÃO: Este é o arquivo principal do programa. É ele que junta o trabalho
4 // de todo o grupo, exibe os menus na tela e direciona o cliente para
5 // a função certa (comprar, cadastrar, sair, etc).
6 // COMPILAÇÃO: gcc main.c usuarios.c catalogo.c carrinho.c checkout.c posvenda.c -o glowup ; ./glowup.exe
7 // =====
8
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12
13 #include "estruturas.h"
14 #include "variaveis_globais.h"
15 #include "usuarios.h"
16 #include "catalogo.h"
17 #include "carrinho.h"
18 #include "checkout.h"
19 #include "posvenda.h"
20
21 // =====
22 // CRIAÇÃO DAS LISTAS DE DADOS
23 // É como se fosse o nosso banco de dados, mas salva na memória ram
24 // =====
25 Usuario usuarios[100];
26 int totalUsuarios = 0;
27
28 Produto catalogo[50];
29 int total_produtos = 0;
30
31 ItemCarrinho carrinho[50];
32 int qtd_itens_carrinho = 0;
33
34 Pedido pedidos[100];

```

```

C main.c > ...
35 int total_pedidos = 0;
36
37 // =====
38 // COLOCAR PRODUTOS INICIAIS NA LOJA
39 // Assim a gente já tem o que vender logo que o programa abre para testar
40 // =====
41 void inicializarSistema()
42 {
43     catalogo[0].id = 1;
44     strcpy(catalogo[0].nome, "Serum Vitamina C");
45     strcpy(catalogo[0].categoria, "Skincare");
46     catalogo[0].preco = 89.90;
47     catalogo[0].estoque = 50;
48
49     catalogo[1].id = 2;
50     strcpy(catalogo[1].nome, "Base Matte Alta Cobertura");
51     strcpy(catalogo[1].categoria, "Maquiagem");
52     catalogo[1].preco = 55.00;
53     catalogo[1].estoque = 30;
54
55     catalogo[2].id = 3;
56     strcpy(catalogo[2].nome, "Espuma de Barbear");
57     strcpy(catalogo[2].categoria, "Cuidados");
58     catalogo[2].preco = 42.50;
59     catalogo[2].estoque = 25;
60
61     catalogo[3].id = 4;
62     strcpy(catalogo[3].nome, "Oleo para Barba e Rosto");
63     strcpy(catalogo[3].categoria, "Cuidados");
64     catalogo[3].preco = 39.90;
65     catalogo[3].estoque = 20;
66
67     catalogo[4].id = 5;
68     strcpy(catalogo[4].nome, "Protetor Solar Toque Seco");

```

```

41 void inicializarSistema()
69     strcpy(catalogo[4].categoria, "Skincare");
70     catalogo[4].preco = 69.90;
71     catalogo[4].estoque = 40;
72
73     catalogo[5].id = 6;
74     strcpy(catalogo[5].nome, "Gel de Limpeza Facial");
75     strcpy(catalogo[5].categoria, "Skincare");
76     catalogo[5].preco = 45.00;
77     catalogo[5].estoque = 35;
78
79     catalogo[6].id = 7;
80     strcpy(catalogo[6].nome, "Hidratante Corporal");
81     strcpy(catalogo[6].categoria, "Cuidados");
82     catalogo[6].preco = 34.90;
83     catalogo[6].estoque = 50;
84
85     catalogo[7].id = 8;
86     strcpy(catalogo[7].nome, "Mascara de Cílios Volume");
87     strcpy(catalogo[7].categoria, "Maquiagem");
88     catalogo[7].preco = 29.90;
89     catalogo[7].estoque = 60;
90
91     catalogo[8].id = 9;
92     strcpy(catalogo[8].nome, "Batom Líquido Vermelho");
93     strcpy(catalogo[8].categoria, "Maquiagem");
94     catalogo[8].preco = 25.50;
95     catalogo[8].estoque = 45;
96
97     catalogo[9].id = 10;
98     strcpy(catalogo[9].nome, "Perfume Floral EDP");
99     strcpy(catalogo[9].categoria, "Perfumaria");

```

```

C main.c > ...
41 void inicializarSistema()
99     strcpy(catalogo[9].categoria, "Perfumaria");
100     catalogo[9].preco = 149.90;
101     catalogo[9].estoque = 15;
102
103     total_produtos = 10;
104
105     // Banner bonitinho de abertura do nosso programa
106     printf("\n");
107     printf("+=====+\n");
108     printf("|                                     |\n");
109     printf("|               BEM-VINDO AO GLOWUP               |\n");
110     printf("|                                     |\n");
111     printf("|               Sua loja de Beleza e Autocuidado               |\n");
112     printf("|                                     |\n");
113     printf("+=====+\n");
114     printf("| Sistema inicializado com sucesso!               |\n");
115     printf("+=====+\n");
116 }
117
118 // =====
119 // CUIDAR DO LOGIN E LOGOUT (Lógica da Sthefanny)
120 // Função para o cliente sair da conta de forma segura
121 // =====
122 void efetuarLogout(int *usuario_Logado)
123 {
124     // Tira a conta da pessoa do sistema
125     *usuario_Logado = -1;
126
127     // Esvazia o carrinho para a próxima pessoa que for usar o sistema não ver as compras dos outros
128     qtd_itens_carrinho = 0;
129     memset(carrinho, 0, sizeof(carrinho));
130
131     printf("\n>> [Sessão Encerrada] Logout efetuado. Carrinho de compras esvaziado por segurança!\n");

```

```

C main.c > ...
122 void efetuarLogout(int *usuario_logado)
123 }
124
125 // =====
126 // FUNÇÃO PRINCIPAL (Onde o programa começa a rodar de verdade)
127 // =====
128 int main()
129 {
130     int opcao;
131     int usuario_logado = -1; // Funciona como uma chave: se for -1, é visitante. Se for número, é cliente logado.
132
133     inicializarSistema();
134
135     // Loop infinito para manter o programa sempre rodando até a pessoa escolher a opção de sair
136     while (1)
137     {
138         // Se a chave for -1, mostramos a tela inicial de quem não está na conta
139         if (usuario_logado == -1)
140         {
141             // MENU INICIAL PARA VISITANTES (Design da Yasmin)
142             printf("\n");
143             printf("+=====+\n");
144             printf("|          GLOWUP E-COMMERCE          |\n");
145             printf("+=====+\n");
146             printf("| 1. Criar Conta                      |\n");
147             printf("| 2. Fazer Login                      |\n");
148             printf("| 0. Sair do Sistema                  |\n");
149             printf("+=====+\n");
150             printf("Escolha uma opcao: ");
151
152             // Proteção feita pela Ana e pelo Marcelo!
153             // O comando 'scanf' tenta ler um número. Se a pessoa digitar uma letra sem querer,
154             // a gente pega o erro aqui para o programa não travar.
155             if (scanf("%d", &opcao) != 1)

```

```

C main.c X
C main.c > ...
165     {
166         printf("\n>> Erro: Digite apenas os numeros do menu.\n");
167         while (getchar() != '\n'); // Limpa a sujeira do teclado
168         continue;
169     }
170     while (getchar() != '\n');
171
172     switch (opcao)
173     {
174         case 1:
175             cadastrarUsuario();
176             break;
177         case 2:
178             // Tenta fazer o login e muda o estado da nossa chave
179             usuario_logado = realizarLogin();
180             if (usuario_logado != -1)
181             {
182                 printf("\n>> Bem-vindo de volta, %s!\n", usuarios[usuario_logado].nome);
183             }
184             break;
185         case 0:
186             printf("\n>> Obrigado por usar o GlowUP! Encerrando...\n");
187             return 0; // Fecha o programa
188         default:
189             printf("\n>> Opcao invalida.\n");
190     }
191
192     else
193     {
194         // Se a pessoa fez login, mostramos o menu com as coisas de comprar
195         printf("\n");
196         printf("+=====+\n");
197         printf("|          PAINEL DO CLIENTE          |\n");
198         printf("+=====+\n");

```

```

C main.c X
C main.c > ...
195 printf("\n");
196 printf("=====+\\n");
197 printf("| PAINEL DO CLIENTE |\\n");
198 printf("=====+\\n");
199 printf("1. Ver Catalogo |\\n");
200 printf("2. Buscar Produto |\\n");
201 printf("3. Ver Carrinho / Adicionar |\\n");
202 printf("4. Finalizar Pedido |\\n");
203 printf("5. Meus Pedidos / Avaliar |\\n");
204 printf("6. Editar Meu Perfil |\\n");
205 printf("0. Deslogar |\\n");
206 printf("=====+\\n");
207 printf("Escolha uma opcao: ");
208
209 // Mesma proteção para não travar na hora de escolher a opção
210 if (scanf("%d", &opcao) != 1)
211 {
212     printf("\\n>> Erro: Digite apenas os numeros do menu.\\n");
213     while (getchar() != '\\n');
214     continue;
215 }
216 while (getchar() != '\\n');
217
218 switch (opcao)
219 {
220     case 1:
221         listarProdutos();
222         break;
223     case 2:
224         buscarProdutoPorNome();
225         break;
226     case 3:
227         exhibirCarrinho();
228         break;

```

```

C main.c > ...
229     case 4:
230     {
231         // Só deixa ir para a tela de pagamento se tiver algo no carrinho!
232         if (calcularSubtotalCarrinho() > 0)
233         {
234             char end[200];
235
236             // Pede o endereço até o cliente digitar do jeito que a gente pediu
237             while (1)
238             {
239                 printf("\\nEndereco (Rua X, Cidade, Brasil): ");
240                 scanf("%199[\\n]", end);
241
242                 while (getchar() != '\\n');
243
244                 if (!validarEndereco(end))
245                 {
246                     printf(">> Erro: Formato invalido. Use: Rua X, Cidade, Brasil.\\n");
247                     continue;
248                 }
249
250                 break;
251             }
252
253             // Calcula todos os valores separadinhos
254             float subtotal = calcularSubtotalCarrinho();
255             float frete = calcularFrete(subtotal);
256             float desconto = aplicarCupom(subtotal);
257             float total = subtotal + frete - desconto;
258
259             // Mostra o extrato da compra para o cliente ver
260             printf("\\n");
261             printf("=====+\\n");
262             printf("| RESUMO DO PEDIDO |\\n");

```

```

C main.c > ...
260 printf("\n");
261 printf("+++++++\n");
262 printf("|          RESUMO DO PEDIDO          |\n");
263 printf("+++++++\n");
264
265 printf("| Subtotal : R$ %-35.2f |\n", subtotal);
266 printf("| Frete    : R$ %-35.2f |\n", frete);
267 printf("| Desconto : R$ %-35.2f |\n", desconto);
268
269 printf("-----+\n");
270
271 printf("| TOTAL    : R$ %-35.2f |\n", total);
272
273 printf("+++++++\n");
274
275 // Se a simulação do pagamento der certo, salva o pedido!
276 if (processarPagamento(total))
277 {
278     faturarPedido(total, usuario_Logado);
279 }
280 else
281 {
282     printf("\n>> Pedido cancelado.\n");
283 }
284
285 else
286 {
287     printf("\n>> Carrinho vazio.\n");
288 }
289 break;
290
291 case 5:
292     menu_pos_venda();

```

```

C main.c > ...
290 break;
291
292 case 5:
293     menu_pos_venda();
294     break;
295
296 case 6:
297     editarDados(usuario_Logado);
298     break;
299 case 0:
300     efetuarLogout(&usuario_Logado); // Sai da conta e limpa o carrinho
301     break;
302 default:
303     printf("\n>> Opcao invalida.\n");
304 }
305
306 }
307 return 0;
308 }

```

9. CÓDIGO

```
//
=====

// ARQUIVO: main.c

// DESCRIÇÃO: Este é o arquivo principal do programa. É ele que junta o trabalho
//           de todo o grupo, exibe os menus na tela e direciona o cliente para
//           a função certa (comprar, cadastrar, sair, etc).

// COMPILAÇÃO: gcc main.c usuarios.c catalogo.c carrinho.c checkout.c posvenda.c -o
//              glowup ; ./glowup.exe

//
=====

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include "estruturas.h"

#include "variaveis_globais.h"

#include "usuarios.h"

#include "catalogo.h"

#include "carrinho.h"

#include "checkout.h"
```

```
#include "posvenda.h"
```

```
//
```

```
=====
```

```
=====
```

```
// CRIAÇÃO DAS LISTAS DE DADOS
```

```
// É como se fosse o nosso banco de dados, mas salva na memória ram
```

```
//
```

```
=====
```

```
=====
```

```
Usuario usuarios[100];
```

```
int totalUsuarios = 0;
```

```
Produto catalogo[50];
```

```
int total_produtos = 0;
```

```
ItemCarrinho carrinho[50];
```

```
int qtd_itens_carrinho = 0;
```

```
Pedido pedidos[100];
```

```
int total_pedidos = 0;
```

```
//
```

```
=====
```

```
=====
```

```
// COLOCAR PRODUTOS INICIAIS NA LOJA
```

// Assim a gente já tem o que vender logo que o programa abre para testar

//

=====

=====

void inicializarSistema()

{

catalogo[0].id = 1;

strcpy(catalogo[0].nome, "Serum Vitamina C");

strcpy(catalogo[0].categoria, "Skincare");

catalogo[0].preco = 89.90;

catalogo[0].estoque = 50;

catalogo[1].id = 2;

strcpy(catalogo[1].nome, "Base Matte Alta Cobertura");

strcpy(catalogo[1].categoria, "Maquiagem");

catalogo[1].preco = 55.00;

catalogo[1].estoque = 30;

catalogo[2].id = 3;

strcpy(catalogo[2].nome, "Espuma de Barbear");

strcpy(catalogo[2].categoria, "Cuidados");

catalogo[2].preco = 42.50;

catalogo[2].estoque = 25;

catalogo[3].id = 4;

```
strcpy(catalogo[3].nome, "Oleo para Barba e Rosto");
```

```
strcpy(catalogo[3].categoria, "Cuidados");
```

```
catalogo[3].preco = 39.90;
```

```
catalogo[3].estoque = 20;
```

```
catalogo[4].id = 5;
```

```
strcpy(catalogo[4].nome, "Protetor Solar Toque Seco");
```

```
strcpy(catalogo[4].categoria, "Skincare");
```

```
catalogo[4].preco = 69.90;
```

```
catalogo[4].estoque = 40;
```

```
catalogo[5].id = 6;
```

```
strcpy(catalogo[5].nome, "Gel de Limpeza Facial");
```

```
strcpy(catalogo[5].categoria, "Skincare");
```

```
catalogo[5].preco = 45.00;
```

```
catalogo[5].estoque = 35;
```

```
catalogo[6].id = 7;
```

```
strcpy(catalogo[6].nome, "Hidratante Corporal");
```

```
strcpy(catalogo[6].categoria, "Cuidados");
```

```
catalogo[6].preco = 34.90;
```

```
catalogo[6].estoque = 50;
```

```
catalogo[7].id = 8;
```

```

strcpy(catalogo[7].nome, "Mascara de Cílios Volume");

strcpy(catalogo[7].categoria, "Maquiagem");

catalogo[7].preco = 29.90;

catalogo[7].estoque = 60;


catalogo[8].id = 9;

strcpy(catalogo[8].nome, "Batom Líquido Vermelho");

strcpy(catalogo[8].categoria, "Maquiagem");

catalogo[8].preco = 25.50;

catalogo[8].estoque = 45;


catalogo[9].id = 10;

strcpy(catalogo[9].nome, "Perfume Floral EDP");

strcpy(catalogo[9].categoria, "Perfumaria");

catalogo[9].preco = 149.90;

catalogo[9].estoque = 15;


total_produtos = 10;


// Banner bonitinho de abertura do nosso programa

printf("\n");

printf("+=====+
==+\n");

printf("|                               |\n");

```

```

printf("|          BEM-VINDO AO GLOWUP          |\n");

printf(|\n");

printf("|      Sua loja de Beleza e Autocuidado      |\n");

printf(|\n");

printf("+=====+
==+\n");

printf("| Sistema inicializado com sucesso!          |\n");

printf("+=====+
==+\n");

}

//
=====

=====

// CUIDAR DO LOGIN E LOGOUT (Lógica da Sthefanny)

// Função para o cliente sair da conta de forma segura

//
=====

=====

void efetuarLogout(int *usuario_logado)

{

    // Tira a conta da pessoa do sistema

    *usuario_logado = -1;

    // Esvazia o carrinho para a próxima pessoa que for usar o sistema não ver as compras dos
    outros

```

```

    qtd_itens_carrinho = 0;

    memset(carrinho, 0, sizeof(carrinho));

    printf("\n>> [Sessao Encerrada] Logout efetuado. Carrinho de compras esvaziado por
seguranca!\n");
}

//
=====

=====

// FUNÇÃO PRINCIPAL (Onde o programa começa a rodar de verdade)

//
=====

=====

int main()

{

    int opcao;

    int usuario_logado = -1; // Funciona como uma chave: se for -1, é visitante. Se for número,
é cliente logado.

    inicializarSistema();

    // Loop infinito para manter o programa sempre rodando até a pessoa escolher a opção de
sair

    while (1)

    {

```

```

// Se a chave for -1, mostramos a tela inicial de quem não está na conta

if (usuario_logado == -1)

{

    // MENU INICIAL PARA VISITANTES (Design da Yasmin)

    printf("\n");

    printf("=====
=====+\n");

    printf("|          GLOWUP E-COMMERCE          |\n");

    printf("=====
=====+\n");

    printf("| 1. Criar Conta                        |\n");

    printf("| 2. Fazer Login                        |\n");

    printf("| 0. Sair do Sistema                    |\n");

    printf("=====
=====+\n");

    printf("Escolha uma opcao: ");

    // Proteção feita pela Ana e pelo Marcelo!

    // O comando 'scanf' tenta ler um número. Se a pessoa digitar uma letra sem querer,

    // a gente pega o erro aqui para o programa não travar.

    if (scanf("%d", &opcao) != 1)

    {

        printf("\n>> Erro: Digite apenas os numeros do menu.\n");

        while (getchar() != '\n'); // Limpa a sujeira do teclado

        continue;

```

```

}

while (getchar() != '\n');

switch (opcao)
{
case 1:

    cadastrarUsuario();

    break;

case 2:

    // Tenta fazer o login e muda o estado da nossa chave

    usuario_logado = realizarLogin();

    if (usuario_logado != -1)

    {

        printf("\n>> Bem-vindo de volta, %s!\n", usuarios[usuario_logado].nome);

    }

    break;

case 0:

    printf("\n>> Obrigado por usar o GlowUP! Encerrando...\n");

    return 0; // Fecha o programa

default:

    printf("\n>> Opcao invalida.\n");

}

}

else

```

```

{

    // Se a pessoa fez login, mostramos o menu com as coisas de comprar

    printf("\n");

    printf("+=====+
=====+ \n");

    printf("|          PAINEL DO CLIENTE          | \n");

    printf("+=====+
=====+ \n");

    printf("| 1. Ver Catalogo                      | \n");

    printf("| 2. Buscar Produto                    | \n");

    printf("| 3. Ver Carrinho / Adicionar          | \n");

    printf("| 4. Finalizar Pedido                  | \n");

    printf("| 5. Meus Pedidos / Avaliar            | \n");

    printf("| 6. Editar Meu Perfil                 | \n");

    printf("| 0. Deslogar                         | \n");

    printf("+=====+
=====+ \n");

    printf("Escolha uma opcao: ");

    // Mesma proteção para não travar na hora de escolher a opção

    if (scanf("%d", &opcao) != 1)

    {

        printf("\n>> Erro: Digite apenas os numeros do menu.\n");

        while (getchar() != '\n');

        continue;

```

```
}

while (getchar() != '\n');


switch (opcao)

{

case 1:

    listarProdutos();

    break;

case 2:

    buscarProdutoPorNome();

    break;

case 3:

    exibirCarrinho();

    break;

case 4:

{

    // Só deixa ir para a tela de pagamento se tiver algo no carrinho!

    if (calcularSubtotalCarrinho() > 0)

    {

        char end[200];


        // Pede o endereço até o cliente digitar do jeito que a gente pediu

        while (1)

        {
```

```

printf("\nEndereco (Rua X, Cidade, Brasil): ");

scanf("%199[^\n]", end);

while (getchar() != '\n');

if (!validarEndereco(end))
{
    printf(">> Erro: Formato invalido. Use: Rua X, Cidade, Brasil.\n");

    continue;
}

break;
}

// Calcula todos os valores separadinhos
float subtotal = calcularSubtotalCarrinho();

float frete = calcularFrete(subtotal);

float desconto = aplicarCupom(subtotal);

float total = subtotal + frete - desconto;

// Mostra o extrato da compra para o cliente ver

printf("\n");

printf("=====
=====+\n");

```

```

printf("|          RESUMO DO PEDIDO          |\n");

printf("+=====+
=====+\n");

printf("| Subtotal : R$ %-35.2f|\n", subtotal);

printf("| Frete   : R$ %-35.2f|\n", frete);

printf("| Desconto : R$ %-35.2f|\n", desconto);

printf("+-----+\n");

printf("| TOTAL   : R$ %-35.2f|\n", total);

printf("+=====+
=====+\n");

// Se a simulação do pagamento der certo, salva o pedido!
if (processarPagamento(total))
{
    faturarPedido(total, usuario_logado);
}
else
{
    printf("\n>> Pedido cancelado.\n");
}
}

```

```
    else

    {

        printf(">> Carrinho vazio.\n");

    }

}

break;


case 5:

    menu_pos_venda();

    break;


case 6:

    editarDados(usuario_logado);

    break;


case 0:

    efetuarLogout(&usuario_logado); // Sai da conta e limpa o carrinho

    break;


default:

    printf("\n>> Opcao invalida.\n");

}

}

}

return 0;

}
```

10. MANUAL DO USUÁRIO - SISTEMA GLOWUP (E-COMMERCE)

10.1. Visão Geral do Sistema

O **GlowUp** é uma plataforma de e-commerce voltada para a venda de cosméticos e produtos de autocuidado. O sistema foi projetado para oferecer uma experiência de navegação rápida e segura pelo terminal, gerenciando desde o seu cadastro inicial até a avaliação dos produtos entregues.

10.1.1. Como Executar (Primeiros Passos)

Para rodar o sistema, você precisará de um terminal com o compilador GCC instalado.

1. Navegue até a pasta raiz do projeto.
2. Compile o código executando o comando: `gcc main.c usuarios.c catalogo.c carrinho.c checkout.c posvenda.c -o glowup`
3. Execute o sistema:
 - **Windows:** `.\glowup.exe`
 - **Linux/Mac:** `./glowup`

10.2. Acesso ao Sistema e Segurança

Ao abrir o GlowUp, você será recebido pelo Menu Principal deslogado, onde poderá criar uma conta, fazer login ou selecionar a opção "0" para encerrar a aplicação com segurança.

10.2.1. Criando sua Conta (Opção 1)

Para realizar compras, é obrigatório possuir um cadastro. O sistema é rigoroso e exigirá que seus dados respeitem as seguintes regras de negócio:

- **Nome:** O sistema não aceita números, você deve digitar apenas letras.
- **Idade:** O e-commerce é restrito para usuários entre 16 e 125 anos.
- **Senha:** Deve conter no mínimo 8 caracteres, mesclando obrigatoriamente letras maiúsculas, minúsculas, números e caracteres especiais.

- **E-mail:** Precisa obrigatoriamente conter um "@" e a extensão ".com". Além disso, não é permitido cadastrar um e-mail que já exista no banco de dados.
- **CPF e Telefone:** Devem ser digitados apenas com números. O CPF requer exatamente 11 dígitos, e o telefone exige pelo menos 11 dígitos. Assim como o e-mail, o sistema impede CPFs duplicados.
- **Endereço Logístico:** Para cálculo de rotas futuras, o formato exigido é: Rua [Nome], [Cidade], Brasil (contendo obrigatoriamente as palavras "Rua", "Brasil" e pelo menos duas vírgulas).

10.2.2. Fazer Login (Opção 2)

Insira seu e-mail e senha cadastrados. **Atenção:** Caso você erre os dados por 3 vezes consecutivas, a sua conta será bloqueada permanentemente na sessão atual por medidas de segurança.

10.3. Navegando e Comprando (Painel do Cliente)

Com o login efetuado, o painel do cliente será desbloqueado.

- **Ver Catálogo (Opção 1):** Exibe a vitrine com IDs, nomes, preços e quantidades disponíveis. Produtos sem estoque são ocultados automaticamente.
- **Buscar Produto (Opção 2):** Permite pesquisar um cosmético pelo nome ou pela categoria (ex: "Skincare"). A busca ignora diferenças entre letras maiúsculas e minúsculas.
- **Ver/Adicionar ao Carrinho (Opção 3):** Insira o ID do produto e a quantidade. O sistema possui um *carrinho inteligente*:
 - Ele verifica em tempo real se a quantidade solicitada existe no estoque.
 - Se você adicionar um produto que já está no seu carrinho, o sistema não duplica o item, apenas soma a nova quantidade à quantidade existente e recalcula o valor automaticamente.

10.4.. Checkout (Finalizando o Pedido)

Acesse a **Opção 4. Finalizar Pedido**. Se o seu carrinho estiver vazio, o sistema bloqueará a ação informando que não é possível prosseguir.

1. **Confirmação de Endereço:** Informe o endereço de entrega, respeitando o mesmo padrão exigido no cadastro ("Rua" e "Brasil").
2. **Frete e Cupons:** Compras de R\$ 150,00 ou mais têm frete grátis; para valores menores, a taxa fixa é de R\$ 19,90. Use o código **GLOW10** para ganhar 10% de desconto no valor dos produtos.
3. **Pagamento:** Selecione PIX, Boleto ou Cartão. Ao confirmar, a compra é efetivada, um número de pedido (a partir de 100) é gerado e as unidades compradas são imediatamente descontadas do estoque da loja.

10.5. Pós-Venda e Edição de Perfil

- **Meus Pedidos / Avaliar (Opção 5):**
 - *Acompanhar Entrega:* Digite o ID do pedido para consultar seu status de logística.
 - *Avaliar Produto:* Se o pedido estiver como "Entregue", você poderá registrar uma nota (1 a 5) e um comentário. Para evitar spam, o sistema garante que um pedido só pode ser avaliado uma única vez.
- **Editar Meu Perfil (Opção 6):** Altere seu endereço e redefina sua senha, passando pelas mesmas verificações de segurança do cadastro. O sistema não permitirá que você coloque uma senha ou endereço idênticos aos atuais.

10.6. Notas Técnicas e Sistema Anti-Crash

- **Deslogar Seguro (Opção 0):** Ao optar por deslogar, o sistema conta com uma limpeza de segurança que esvazia qualquer item que tenha ficado no seu carrinho, protegendo as informações para o próximo usuário.
- **Blindagem de Menus (Buffer Overflow):** Se ao navegar pelos menus (que exigem números) você digitar acidentalmente uma letra, a aplicação não irá fechar ou travar. A interface absorverá o erro, limpará a memória e exibirá o aviso "*Erro: Digite apenas os numeros do menu*", pedindo uma nova escolha.

PLANNER DOCUMENTAÇÃO

Primeira versão do planner – 12/02 – 21/02

The screenshot shows the first version of the documentation planner. It features a dark red header with the CEUMA logo and the title 'Planner'. Below the header, there are navigation tabs: 'Meus planos', 'Backlog: Projeto GlowUp', 'Grade', 'Quadro', 'Agenda', and 'Gráficos'. The main area is divided into five columns, each representing a different phase of the project. Each column has a title and a '+ Adicionar tarefa' button. The tasks are listed in a grid, with some tasks having a progress indicator (a circle with a number) and a date. The 'Reuniões' column has two sub-sections, 'Reuniões' and 'Reuniões'. The 'Escopo e Definições' column has two sub-sections, 'Definições de Neg...' and 'Definições de Neg...'. The 'Requisitos e Regras' column has two sub-sections, 'Documentação Tec...' and 'Requisitos'. The 'Modelagem de Usuário' column has two sub-sections, 'Documentação Tec...' and 'User Stories (HUs)'. The 'Dinâmica e Fluxos (Sequência)' column has two sub-sections, 'Lógica de Fluxo (Se...' and 'Lógica de Fluxo (Se...'.

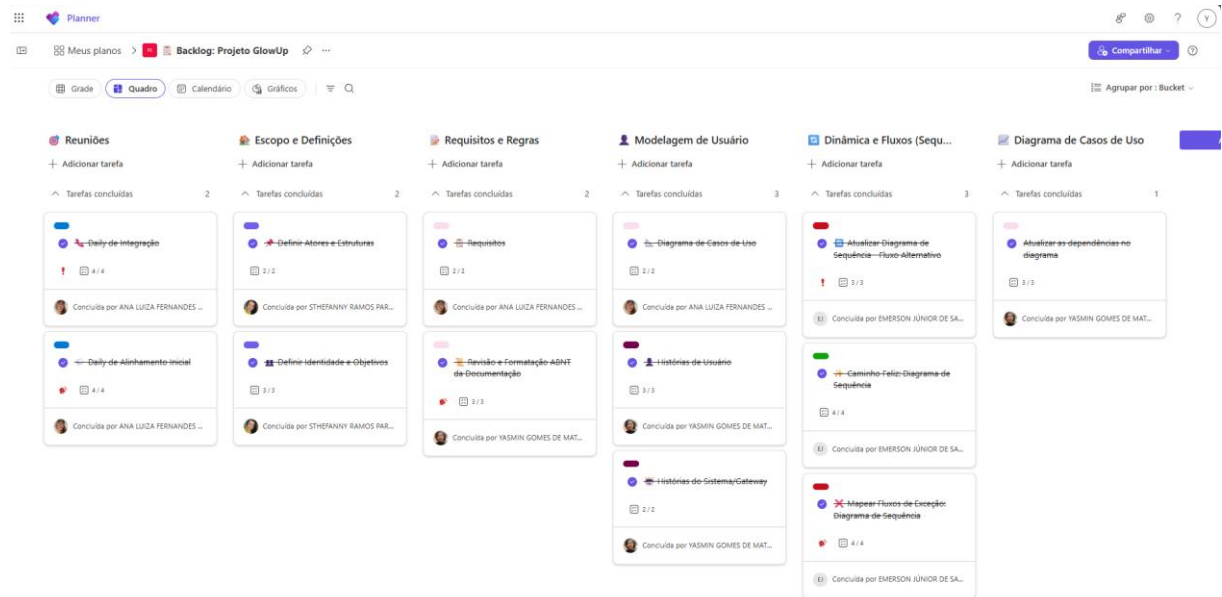
Fonte: Elaborado pelos autores (2026).

Segunda versão do planner - 21/02 – 24/02

The screenshot shows the second version of the documentation planner. It features a dark red header with the CEUMA logo and the title 'Planner'. Below the header, there are navigation tabs: 'Meus planos', 'Backlog: Projeto GlowUp', 'Grade', 'Quadro', 'Calendário', and 'Gráficos'. The main area is divided into five columns, each representing a different phase of the project. Each column has a title and a '+ Adicionar tarefa' button. The tasks are listed in a grid, with some tasks having a progress indicator (a circle with a number) and a date. The 'Reuniões' column has two sub-sections, 'Reuniões' and 'Reuniões'. The 'Escopo e Definições' column has two sub-sections, 'Negócio' and 'Negócio'. The 'Requisitos e Regras' column has two sub-sections, 'Documentação ...' and 'Documentação ...'. The 'Modelagem de Usuário' column has two sub-sections, 'Documentação ...' and 'User Stories (HUs)'. The 'Dinâmica e Fluxos (Sequência)' column has two sub-sections, 'Fluxo Alternativ...' and 'Fluxo Alternativ...'.

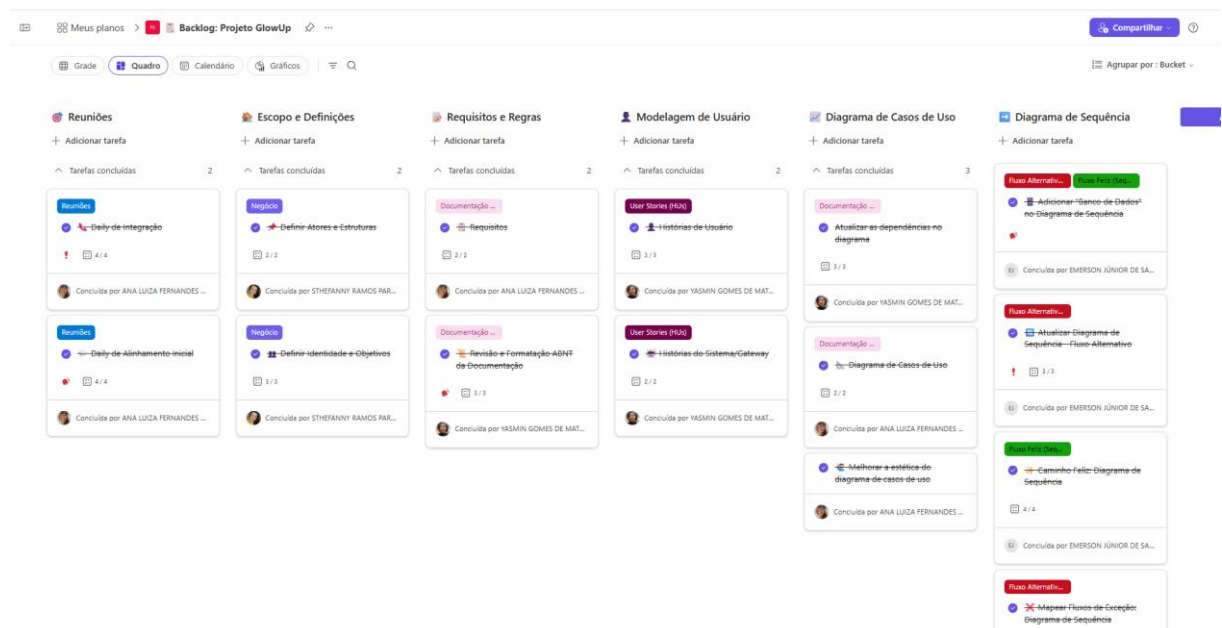
Fonte: Elaborado pelos autores (2026).

Terceira versão do planner - 21/02 – 24/03



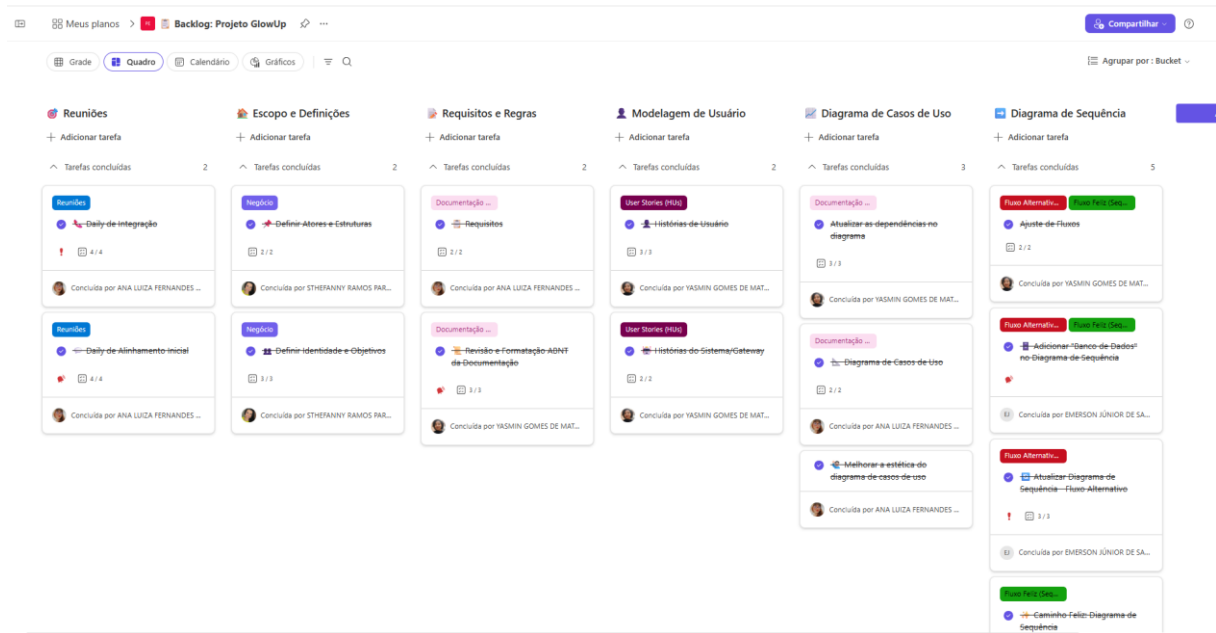
Fonte: Elaborado pelos autores (2026).

Quarta versão do planner - 24/03 – 30/03



Fonte: Elaborado pelos autores (2026).

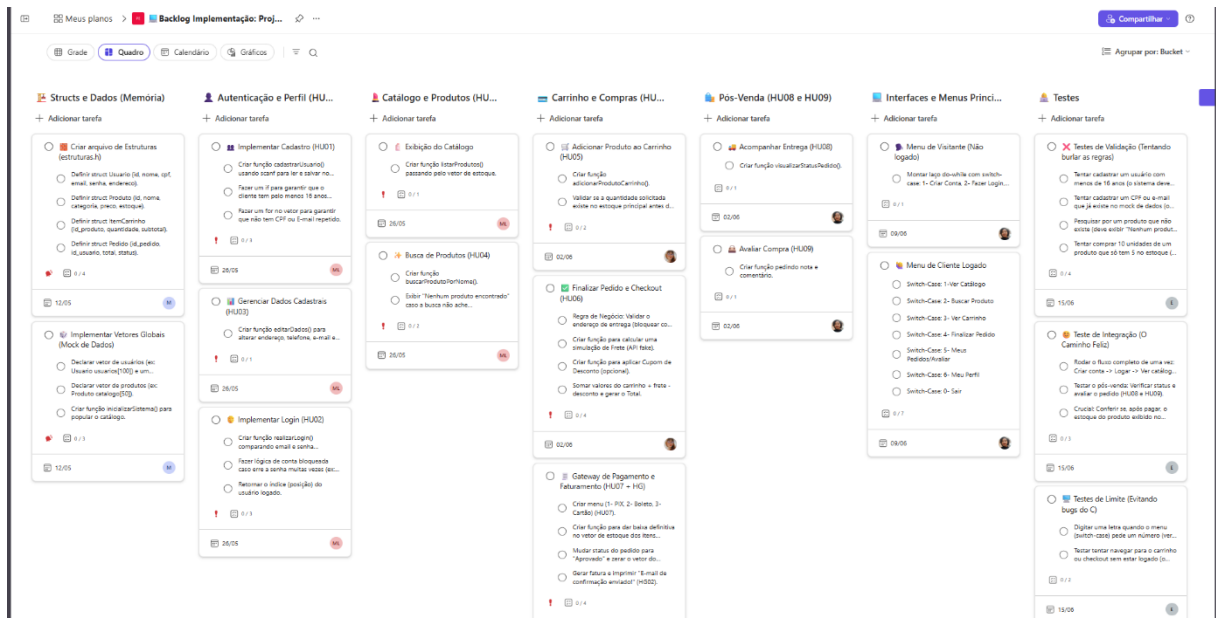
Última versão do planner com todas as tarefas concluídas – 31/03



Fonte: Elaborado pelos autores (2026).

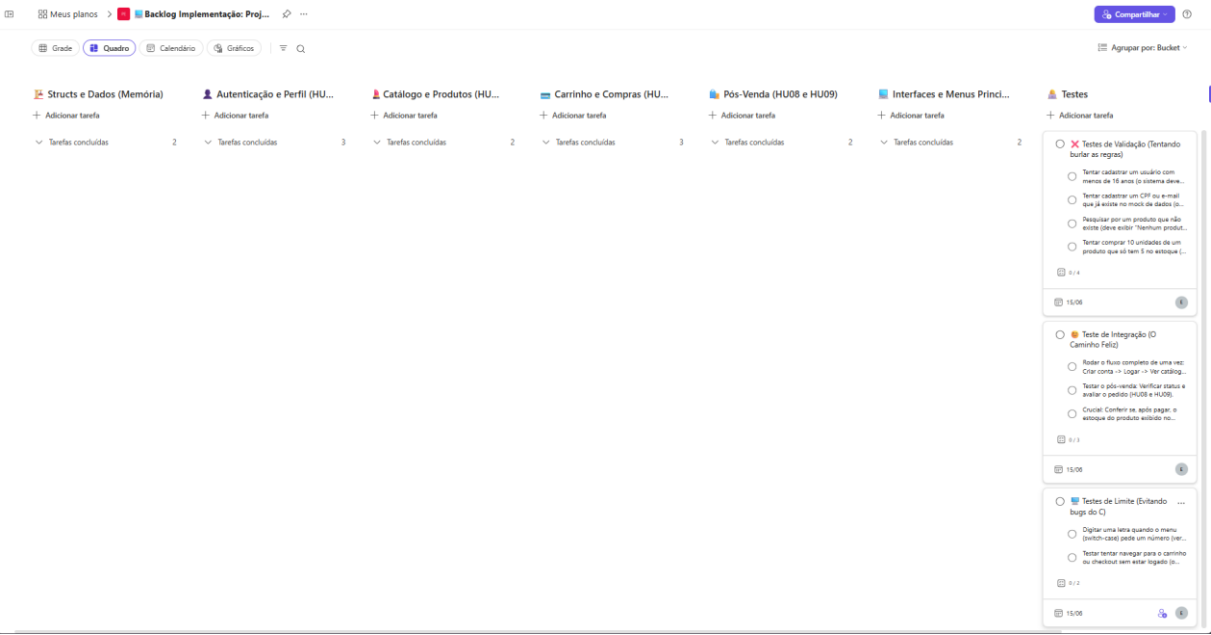
PLANNER CODIFICAÇÃO DO SISTEMA

Primeira versão do planner – 05/05



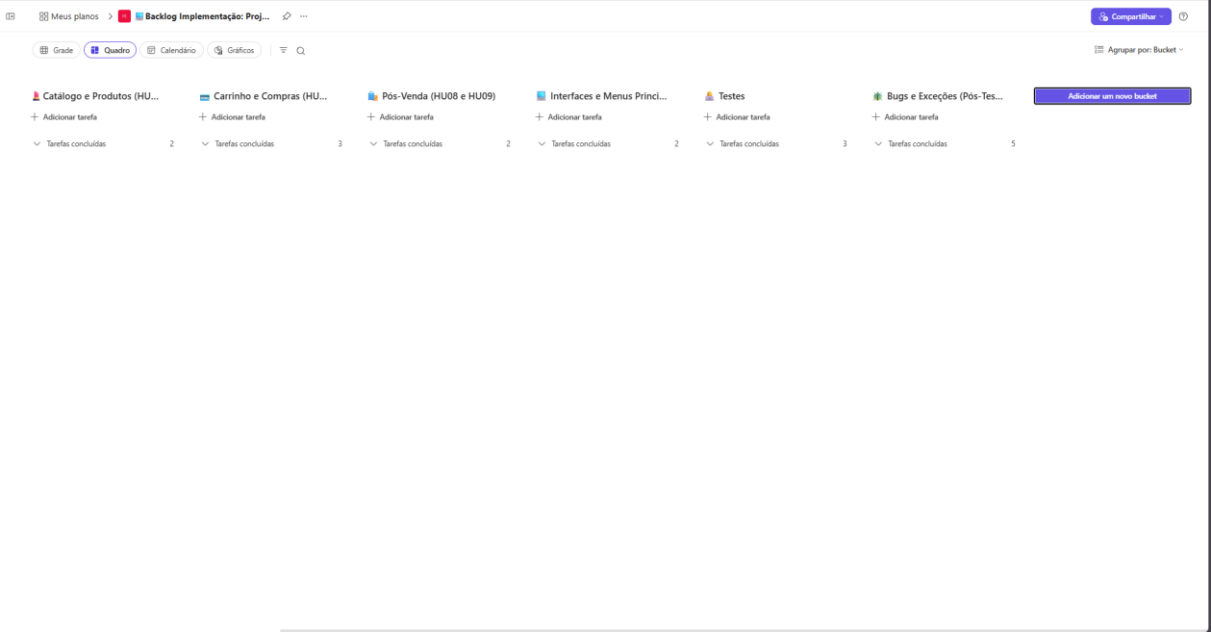
Fonte: Elaborado pelos autores (2026).

Segunda versão do planner – 05/05 – 27/05



Fonte: Elaborado pelos autores (2026).

Última versão do planner, com a codificação finalizada– 05/05 – 10/06



Fonte: Elaborado pelos autores (2026).